Muhammad Jehanzaib

AI Case Studies — Architecture, Agents & Voice

In-depth engineering case studies of products I designed and built: the problem, the requirements, the architecture, the key decisions, and measured results.

mjehanzaib99@gmail.com · linkedin.com/in/muhammad-jehanzaib-4036b5195 · github.com/mjehanzaib999 · Resume: mjehanzaib-ai-resume.up.railway.app · Portfolio: mjehanzaib-ai-portfolio.up.railway.app

LIVE deployed & clickable · DEMO video walkthrough · FLAGSHIP lead production work

Case studies

01 Omvi William — Multi-Tenant Agentic AI SaaS with Real-Time Voice — Conversational AI / Telecom

01 · Omvi William — Multi-Tenant Agentic AI SaaS with Real-Time Voice FLAGSHIP DEMO

Conversational AI / Telecom

Client: World Mobile (telecom operator), delivered through Omvi AI · **Role:** architect and engineer, designed and built end to end · **Timeline:** Oct 2025 – Mar 2026, demo platform Oct 2026 · **Demo:** Watch the walkthrough ↗

A white-labelled AI assistant, "William," that people can talk to or type to. It answers from the operator's own knowledge base, acts in email and calendar, remembers earlier conversations, guides new users through onboarding, and turns a voice call into a project that a team of five AI agents completes while the user watches. One agent brain serves both chat and voice, and the platform is built to serve many clients from one codebase.

2.0 s end of question → agent starts speaking (median, 9 live calls)	**3.9 s** end of question → grounded knowledge answer starts	**35 → <10 s** first reply after a cold start	**15–20 s** per task through all five agents
6 services around one agent brain	**15 + 6** LangGraph nodes in the chat and voice graphs	**25** agent tools (12 email, 7 calendar, 6 general)	**6** RBAC roles in the multi-tenant model

The problem

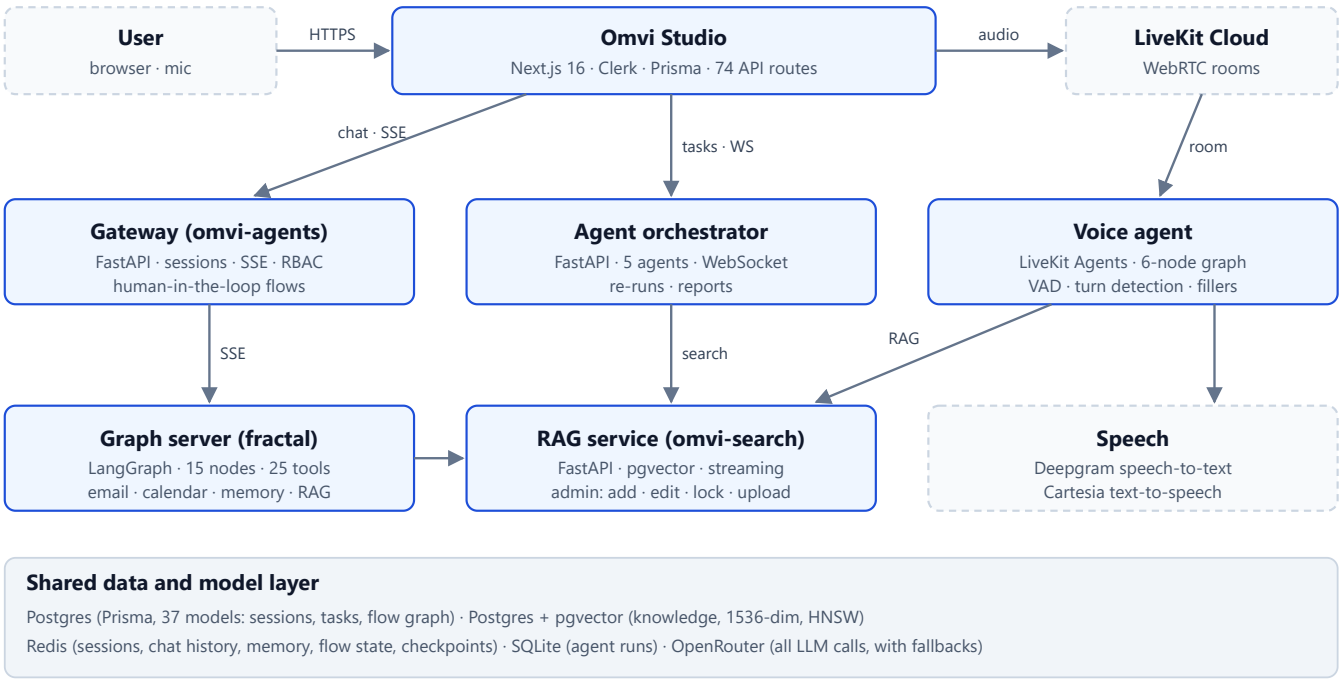
A telecom operator wanted one assistant its customers and staff could use both by voice and by text, grounded in the company's own material rather than the open web, able to take actions (email, calendar, tasks) rather than only answer, and good enough on a live call that it never feels like a slow bot. The vendor building it, Omvi AI, also needed the product to be sellable to the next telecom or enterprise client without rebuilding it for each one.

Requirements I derived and built against

Requirement	How the system meets it
Talk or type, and get the same answers	One LangGraph agent behind both: chat streams over SSE through the gateway; voice runs the same reasoning inside a LiveKit worker.
Answer from the operator's knowledge, not the web	pgvector RAG over the company corpus with neighbouring-passage context and a similarity floor; says so plainly when nothing relevant is found.
Take actions in email and calendar	Dedicated intent resolvers with 12 email and 7 calendar tools over Google and Microsoft OAuth; drafts are confirmed before anything is sent.
Remember the user	Temporal and cross-session memory recall nodes over Redis chat history, with safeguards against fabricated recall.
Guide new users	Human-in-the-loop onboarding and daily-reminder flows that pause for the user with LangGraph `interrupt()` and resume where they left off.
Turn conversation into work	A call transcript becomes a titled session with up to 20 tasks across 8 lanes and their dependencies; five agents then complete each task.
Feel instant on a phone-style call	Under 2.5 s from the end of a question to the agent speaking, measured on the deployed system.
Serve many clients from one codebase	Organization-scoped state, 6 roles mapped to `module:action` scopes, per-endpoint scope checks and a cross-organization guard.
Let admins control the knowledge	Knowledge entries can be added, edited, locked against changes, deleted, or uploaded as PDF, DOCX, HTML or text.
Show cost and progress	Per-agent output, timing and token cost; a live usage dashboard; notifications when agents finish.

Architecture

I split the product into six services so the agent, the voice runtime and the task agents can each scale and ship on their own, while sharing one brain and one data layer.



Six services (blue) around one agent brain; dashed boxes are external. Chat flows Studio → gateway → graph server; voice flows LiveKit → voice agent; tasks flow Studio → orchestrator, with progress pushed back over WebSocket.

- **Omvi Studio** (Next.js 16, Clerk, Prisma): the workspace. Sessions, chat, the voice call screen with a live transcript, Board / Grid / Canvas views of tasks, email and calendar cards, dashboard, knowledge admin, notifications and search.
- **Gateway** (FastAPI): the single stateful entry point for every chat and voice request. Redis sessions and history, SSE streaming to the client, role and scope checks, and LangGraph flows that pause for user input.
- **Graph server** (FastAPI + LangGraph): the agent brain, split out of the gateway so it ships independently. Streams tokens back to the gateway over SSE.
- **Voice agent** (LiveKit Agents): joins each call room, listens, reasons with its own LangGraph graph and speaks; kept warm so it joins quickly.
- **RAG service** (FastAPI + pgvector): retrieval and grounded answers, streamed or whole, plus the knowledge admin API.
- **Agent orchestrator** (FastAPI): watches for new tasks and runs each through five agents, pushing progress to the Studio over WebSocket and storing every output and report.

Agent design

The graph server's LangGraph agent has 15 nodes. A **main orchestrator** decides each turn: email and calendar requests go to their own **decision → intent resolver → action** chains (the resolvers bind 12 email and 7 calendar tools, and a tool interpreter turns raw tool results into a reply); everything else goes through an **LLM domain classifier** and a **normal decision** node to knowledge retrieval, **temporal or cross-session memory recall**, or the **speaker**, a tool-calling responder bound to knowledge search, web search, time and task creation.

- **Human in the loop:** onboarding, daily-reminder and content flows (7, 6 and 3 nodes) pause with `interrupt()` and resume with `Command(resume=…)`, checkpointed in Redis, so a user can answer days later and pick up mid-flow.
- **Streaming end to end:** only the answer-writing model calls are tagged for the user; routers and classifiers stay internal. On Python 3.10 nested LLM calls only reach the graph's event stream when a node hands down its run config, so each answer node passes it explicitly, and the RAG service's own answer stream is relayed through the graph as custom events.
- **Resilience:** every model call goes through OpenRouter with ordered fallback models and retries, so a rate-limited provider becomes a slower answer, not an error.

Voice pipeline

The voice agent runs on LiveKit Agents: Silero voice activity detection, a multilingual turn-detection model, BVC noise cancellation, Deepgram speech-to-text and Cartesia text-to-speech. Its six-node LangGraph graph (decide, greeting, general response, memory recall, retrieval, content creation) answers every turn, and I patched the LiveKit LangChain adapter so the graph, not the plugin, decides what is spoken.

- **Fast route:** a clear product question goes straight to retrieval without an LLM routing call; requests that need judgement (summaries, drafts, recall) still use the router.
- **No dead air:** a short filler line ("let me check that for you") is spoken the moment retrieval starts, then the answer streams sentence by sentence.
- **Warm workers:** two idle worker processes are kept ready, so the agent joins a call without a cold start.
- **Call to project:** when the caller hangs up, the transcript is sent to the Studio, which creates a titled session and extracts tasks (1 to 20, scaled to the length of the call) with lanes and dependencies.

From conversation to finished work

The orchestrator picks up new tasks as soon as they are written (it also polls the task table every 3 seconds, because voice-created tasks never call its API) and runs up to three at a time through **Omni** (research), **Planner**, **Builder**, **Reviewer** and **Nexus** (write-up). Each stage streams lane transitions to the Studio, stores its output with

timing and input/output tokens in SQLite, and the last stage produces a downloadable report. **Refine** re-runs the agents with a user instruction instead of starting over.

Knowledge (RAG)

Documents are chunked to about 1,800 characters and embedded with `text-embedding-3-small` (1536 dimensions) into Postgres with an HNSW cosine index. Each query retrieves at least 8 passages above a similarity floor and adds the passage that follows each of the top four, so answers keep their surrounding context. Answers are written by Gemini 2.5 Flash Lite with GPT-4.1 mini and GPT-4o mini as fallbacks, either whole or streamed.

Multi-tenancy and platform

- **Tenant isolation:** every request carries an organization, role and scopes; Redis state is keyed `{namespace}:{org}:{user}:{session}`; a middleware guard blocks cross-organization access.
- **Roles and scopes:** six roles (platform admin through client viewer) map to `{module}:{action}` scopes, and an endpoint-to-scope map enforces them. I authored the architecture plans the team adopted for serving many clients from one codebase.
- **Delivery:** shipped to GCP through GitHub Actions, and later as 8 services on Railway (six apps plus Postgres with pgvector and Redis) with auto-deploy on merge and Playwright browser audits before release.

Key engineering decisions

Problem	Decision
Chat and voice drifting apart	One LangGraph brain; the voice worker runs the same reasoning and a patched adapter lets the graph control speech.
Silence while the agent thinks	Skip the router for clear product questions, speak a filler immediately, stream the answer by sentence.
Answers arriving all at once	Pass run config into answer nodes, tag user-facing calls, relay the RAG stream as graph events.
Slow first request after deploy	Build and compile the agent graph at startup; first reply went from 35 s to under 10 s.
Provider outages and rate limits	One OpenRouter gateway with ordered fallback models and retries in every service.
Voice-created tasks invisible to the agents	Orchestrator watches the task table directly, in addition to its trigger API.
Each new client needing a fork	Organization-scoped state, role-to-scope permissions and per-endpoint checks from day one.

Results

Voice latency measured on the deployed system on 3 November 2026: nine live calls, each a spoken question played into a browser microphone, timed from the live transcript (which is shown in step with the agent's audio).

Moment	Median	Range
Click "Voice" → agent greets the user	**7.5 s**	7.4 – 10.1 s
End of a knowledge question → agent starts speaking	**2.0 s**	1.7 – 2.6 s
End of a knowledge question → grounded answer starts	**3.9 s**	3.5 – 4.5 s
End of a general question → answer starts	**4.0 s**	2.8 – 4.0 s

- All nine spoken questions were transcribed correctly and answered from the knowledge base, including exact plan prices.
- A task created from a call finishes all five agents in about 15–20 seconds; Refine re-runs in about 13 seconds.
- The first reply after a deploy dropped from 35 s to under 10 s.

Python · LangGraph · LangChain · FastAPI · LiveKit Agents · Deepgram · Cartesia · Silero · PostgreSQL + pgvector · Redis · SQLite · OpenRouter (Gemini 2.5 Flash Lite, GPT-4.1 mini, GPT-4o mini) · SSE · WebSockets · Next.js 16 · TypeScript · Prisma · Clerk · Docker · Railway · GCP · GitHub Actions · Playwright

Watch demo ↗

---

mjehanzaib99@gmail.com · linkedin.com/in/muhammad-jehanzaib-4036b5195 · github.com/mjehanzaib999